Technical Design of a RAG-Based Document QA System

Abstract

This document presents a detailed technical overview of a Retrieval-Augmented Generation (RAG) system. It discusses architecture, document ingestion, chunking strategies, embedding models, vector databases, semantic search, reranking, prompt engineering, evaluation metrics, deployment considerations, scalability, security, and future enhancements.

This document presents a detailed technical overview of a Retrieval-Augmented Generation (RAG) system. It discusses architecture, document ingestion, chunking strategies, embedding models, vector databases, semantic search, reranking, prompt engineering, evaluation metrics, deployment considerations, scalability, security, and future enhancements.

This document presents a detailed technical overview of a Retrieval-Augmented Generation (RAG) system. It discusses architecture, document ingestion, chunking strategies, embedding models, vector databases, semantic search, reranking, prompt engineering, evaluation metrics, deployment considerations, scalability, security, and future enhancements.

This document presents a detailed technical overview of a Retrieval-Augmented Generation (RAG) system. It discusses architecture, document ingestion, chunking strategies, embedding models, vector databases, semantic search, reranking, prompt engineering, evaluation metrics, deployment considerations, scalability, security, and future enhancements.

1. Introduction

RAG combines retrieval systems with large language models. Documents are converted into embeddings, indexed, and searched using semantic similarity before the retrieved context is supplied to the language model.

RAG combines retrieval systems with large language models. Documents are converted into embeddings, indexed, and searched using semantic similarity before the retrieved context is supplied to the language model.

RAG combines retrieval systems with large language models. Documents are converted into embeddings, indexed, and searched using semantic similarity before the retrieved context is supplied to the language model.

RAG combines retrieval systems with large language models. Documents are converted into embeddings, indexed, and searched using semantic similarity before the retrieved context is supplied to the language model.

2. System Architecture

The architecture contains a frontend, FastAPI backend, PDF parser, embedding service, FAISS vector database, retriever, LLM inference engine, and response generator.

The architecture contains a frontend, FastAPI backend, PDF parser, embedding service, FAISS vector database, retriever, LLM inference engine, and response generator.

The architecture contains a frontend, FastAPI backend, PDF parser, embedding service, FAISS vector database, retriever, LLM inference engine, and response generator.

The architecture contains a frontend, FastAPI backend, PDF parser, embedding service, FAISS vector database, retriever, LLM inference engine, and response generator.

3. Document Processing

PDFs are parsed, cleaned, split into overlapping chunks, embedded, and indexed. Metadata such as filename, page number, and section are preserved.

PDFs are parsed, cleaned, split into overlapping chunks, embedded, and indexed. Metadata such as filename, page number, and section are preserved.

PDFs are parsed, cleaned, split into overlapping chunks, embedded, and indexed. Metadata such as filename, page number, and section are preserved.

PDFs are parsed, cleaned, split into overlapping chunks, embedded, and indexed. Metadata such as filename, page number, and section are preserved.

4. Embeddings

Sentence transformers convert text into dense vectors that capture semantic meaning rather than keywords.

Sentence transformers convert text into dense vectors that capture semantic meaning rather than keywords.

Sentence transformers convert text into dense vectors that capture semantic meaning rather than keywords.

Sentence transformers convert text into dense vectors that capture semantic meaning rather than keywords.

5. Vector Database

FAISS performs efficient nearest-neighbor search for millions of vectors using cosine similarity or L2 distance.

FAISS performs efficient nearest-neighbor search for millions of vectors using cosine similarity or L2 distance.

FAISS performs efficient nearest-neighbor search for millions of vectors using cosine similarity or L2 distance.

FAISS performs efficient nearest-neighbor search for millions of vectors using cosine similarity or L2 distance.

6. Retrieval Pipeline

User queries are embedded and matched against indexed chunks. Top-k chunks are optionally reranked before prompting the LLM.

User queries are embedded and matched against indexed chunks. Top-k chunks are optionally reranked before prompting the LLM.

User queries are embedded and matched against indexed chunks. Top-k chunks are optionally reranked before prompting the LLM.

User queries are embedded and matched against indexed chunks. Top-k chunks are optionally reranked before prompting the LLM.

7. Prompt Engineering

The retrieved context is injected into prompts with clear instructions to answer only from the provided documents.

The retrieved context is injected into prompts with clear instructions to answer only from the provided documents.

The retrieved context is injected into prompts with clear instructions to answer only from the provided documents.

The retrieved context is injected into prompts with clear instructions to answer only from the provided documents.

8. Evaluation

Performance is measured using Precision@K, Recall@K, MRR, latency, faithfulness, and answer relevance.

Performance is measured using Precision@K, Recall@K, MRR, latency, faithfulness, and answer relevance.

Performance is measured using Precision@K, Recall@K, MRR, latency, faithfulness, and answer relevance.

Performance is measured using Precision@K, Recall@K, MRR, latency, faithfulness, and answer relevance.

9. Deployment

Docker containers, REST APIs, logging, monitoring, authentication, and GPU acceleration support production deployment.

Docker containers, REST APIs, logging, monitoring, authentication, and GPU acceleration support production deployment.

Docker containers, REST APIs, logging, monitoring, authentication, and GPU acceleration support production deployment.

Docker containers, REST APIs, logging, monitoring, authentication, and GPU acceleration support production deployment.

10. Security

Sensitive documents require encryption, role-based access control, audit logging, and secure API keys.

Sensitive documents require encryption, role-based access control, audit logging, and secure API keys.

Sensitive documents require encryption, role-based access control, audit logging, and secure API keys.

Sensitive documents require encryption, role-based access control, audit logging, and secure API keys.

11. Limitations

Poor OCR quality, ambiguous questions, and weak chunking can reduce retrieval accuracy.

Poor OCR quality, ambiguous questions, and weak chunking can reduce retrieval accuracy.
Poor OCR quality, ambiguous questions, and weak chunking can reduce retrieval accuracy.
Poor OCR quality, ambiguous questions, and weak chunking can reduce retrieval accuracy.

12. Future Work

Hybrid search, knowledge graphs, multimodal retrieval, and agentic workflows improve next-generation RAG systems.

Hybrid search, knowledge graphs, multimodal retrieval, and agentic workflows improve next-generation RAG systems.

Hybrid search, knowledge graphs, multimodal retrieval, and agentic workflows improve next-generation RAG systems.

Hybrid search, knowledge graphs, multimodal retrieval, and agentic workflows improve next-generation RAG systems.

References

Lewis et al. (2020); FAISS documentation; LangChain documentation; Hugging Face sentence transformers; FastAPI documentation.

Lewis et al. (2020); FAISS documentation; LangChain documentation; Hugging Face sentence transformers; FastAPI documentation.

Lewis et al. (2020); FAISS documentation; LangChain documentation; Hugging Face sentence transformers; FastAPI documentation.

Lewis et al. (2020); FAISS documentation; LangChain documentation; Hugging Face sentence transformers; FastAPI documentation.